

Student 36 **VANGIMALLA BALA SIVA PRASAD REDDY** [192424316RA](#)

5 / 5 submitted

Q1. Smart Document Editor using String, StringBuilder and StringBuffer

CODE

```
import java.util.*;

public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        String paragraph = sc.nextLine();
        String oldWord = sc.nextLine();
        String newWord = sc.nextLine();

        int characters = paragraph.length();

        String[] words = paragraph.trim().split("\\s+");
        int wordCount = words.length;

        String[] sentences = paragraph.split("[.!?]+");
        int sentenceCount = 0;
        for (String s : sentences) {
            if (!s.trim().isEmpty())
                sentenceCount++;
        }

        String replaced = paragraph.replace(oldWord, newWord);

        StringBuilder finalDoc = new StringBuilder();
        String[] arr = replaced.split("[.!?]+");

        for (String s : arr) {
            String rev = new StringBuilder(s.trim()).reverse().toString();
            finalDoc.append(rev).append(" ");
        }

        StringBuffer history = new StringBuffer();
        history.append("Original Paragraph\n");
        history.append(paragraph).append("\n");
        history.append("Replaced ").append(oldWord).append("' with '")
            .append(newWord).append("\n");

        System.out.println("Original Document:");
        System.out.println(paragraph);

        System.out.println("Characters : " + characters);
        System.out.println("Words : " + wordCount);
        System.out.println("Sentences : " + sentenceCount);

        System.out.println("Editing History:");
        System.out.println(history);

        System.out.println("Final Document:");
        System.out.println(finalDoc.toString().trim());

    }
}
```

INPUT

Java is easy. Java is powerful.
Java
Python

OUTPUT

(no output)

Q2. Hospital Token Management System using List Interface

CODE

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        List<Patient> patients = new ArrayList<>();

        int n = sc.nextInt();
        sc.nextLine();

        for (int i = 0; i < n; i++) {
            int id = sc.nextInt();
            sc.nextLine();
            String name = sc.nextLine();
            String department = sc.nextLine();

            patients.add(new Patient(id, name, department));
        }

        int updateId = sc.nextInt();
        sc.nextLine();
        String newName = sc.nextLine();
        String newDept = sc.nextLine();

        for (Patient p : patients) {
            if (p.id == updateId) {
                p.name = newName;
                p.department = newDept;
            }
        }

        int removeId = sc.nextInt();
        sc.nextLine();

        patients.removeIf(p -> p.id == removeId);

        String searchDept = sc.nextLine();

        for (Patient p : patients) {
            System.out.println(p.id + " " + p.name + " " + p.department);
        }

        for (Patient p : patients) {
            if (p.department.equalsIgnoreCase(searchDept)) {
                System.out.println(p.id + " " + p.name + " " + p.department);
            }
        }

        if (!patients.isEmpty()) {
            Patient first = patients.get(0);
            Patient last = patients.get(patients.size() - 1);

            System.out.println(first.id + " " + first.name + " " + first.department);
            System.out.println(last.id + " " + last.name + " " + last.department);
        }
    }
}

class Patient {
    int id;
```

Report

```
String name;  
String department;  
  
Patient(int id, String name, String department) {  
    this.id = id;  
    this.name = name;  
    this.department = department;  
}  
}
```

INPUT

```
4  
101  
Rahul  
Cardiology  
102  
Priya  
Neurology  
103  
Amit  
Orthopedics  
104  
Sneha  
Cardiology  
102  
Priya Sharma  
Pediatrics  
103  
Cardiology
```

OUTPUT

(no output)

Q3. Airport Boarding Queue Management using Queue Interface

CODE

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        Queue<String> queue = new LinkedList<>();
        PriorityQueue<String> vipQueue = new PriorityQueue<>();

        int n = sc.nextInt();
        sc.nextLine();

        for (int i = 0; i < n; i++) {
            queue.offer(sc.nextLine());
        }

        for (String p : queue) {
            System.out.println(p);
        }

        if (!queue.isEmpty()) {
            System.out.println(queue.poll());
        }

        if (!queue.isEmpty()) {
            System.out.println(queue.peek());
        }

        int m = sc.nextInt();
        sc.nextLine();

        for (int i = 0; i < m; i++) {
            vipQueue.offer(sc.nextLine());
        }

        while (!vipQueue.isEmpty()) {
            System.out.println(vipQueue.poll());
        }

        for (String p : queue) {
            System.out.println(p);
        }
    }
}
```

INPUT

```
4
Rahul
Priya
Amit
Sneha
2
Vijay
Arun
```

OUTPUT

(no output)

Q4. Digital Certificate Verification using Set Interface

CODE

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        HashSet<String> certificates = new HashSet<>();

        int n = sc.nextInt();
        sc.nextLine();

        for (int i = 0; i < n; i++) {
            certificates.add(sc.nextLine());
        }

        String search = sc.nextLine();
        System.out.println(certificates.contains(search));

        for (String id : certificates) {
            System.out.println(id);
        }

        TreeSet<String> sorted = new TreeSet<>(certificates);
        for (String id : sorted) {
            System.out.println(id);
        }

        System.out.println(certificates.size());

        String revoke = sc.nextLine();
        certificates.remove(revoke);

        for (String id : certificates) {
            System.out.println(id);
        }
    }
}
```

INPUT

```
5
C101
C102
C103
C101
C104
C102
C103
```

OUTPUT

(no output)

Q5. Smart Inventory Management using Map Interface

CODE

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        HashMap<Integer, Product> map = new HashMap<>();

        int n = sc.nextInt();

        for (int i = 0; i < n; i++) {
            int id = sc.nextInt();
            sc.nextLine();
            String name = sc.nextLine();
            String category = sc.nextLine();
            int qty = sc.nextInt();
            double price = sc.nextDouble();

            map.put(id, new Product(id, name, category, qty, price));
        }

        int updateId = sc.nextInt();
        int newQty = sc.nextInt();

        if (map.containsKey(updateId))
            map.get(updateId).qty = newQty;

        int searchId = sc.nextInt();
        if (map.containsKey(searchId))
            System.out.println(map.get(searchId));

        double total = 0;
        for (Product p : map.values())
            total += p.qty * p.price;
        System.out.println(total);

        sc.nextLine();
        String category = sc.nextLine();
        for (Product p : map.values())
            if (p.category.equalsIgnoreCase(category))
                System.out.println(p);

        int removeId = sc.nextInt();
        map.remove(removeId);

        for (Product p : map.values())
            System.out.println(p);
    }
}

class Product {
    int id, qty;
    String name, category;
    double price;

    Product(int id, String name, String category, int qty, double price) {
        this.id = id;
        this.name = name;
        this.category = category;
        this.qty = qty;
        this.price = price;
    }
}
```

Report

```
    }

    public String toString() {
        return id + " " + name + " " + category + " " + qty + " " + price;
    }
}
```

INPUT

3
101
Laptop
Electronics
5
50000
102
Mouse
Electronics
10
500
103
Chair
Furniture
4
2000
102
15
101
Electronics
103

OUTPUT

(no output)